

保险营销内容智能审核系统 — Demo→生产差异清单

版本: v 3 . 5 | 更新日期: 2 0 2 6 - 0 5 - 1 7

一、概述

1.1 文档目的

本文档系统性地梳理当前Demo版本与生产环境要求之间的差距，为团队提供清晰的迁移路线图。Demo版本以功能验证为核心目标，在数据存储、可靠性、安全性、可扩展性等方面存在大量简化实现，无法直接承载生产级流量和合规要求。

1.2 Demo版本现状

Demo版本基于以下技术栈运行：

维度	Demo实现
数据存储	SQLite单文件数据库（insurance_review.db）
向量数据库	ChromaDB本地持久化（./data/vector_store）
LLM调用	单API Key直连阿里云DashScope
前端界面	纯HTML单页应用（FastAPI后端）
部署方式	单机 python api_server.py 启动
认证鉴权	单一admin角色，硬编码默认密码 admin123
监控告警	基础日志 + 可选Prometheus指标
安全防护	内存级Rate Limiter + 3 3 种安全检测(2 5 注入+ 5 XSS+ 3 SQLi)

1.3 生产环境核心要求

- 高可用：**服务可用性 $\geq 99.9\%$ ，单点故障自动切换
- 高性能：**P99延迟 $\leq 3s$ ，支持100+QPS并发审核
- 安全合规：**通过等保三级评估，满足金融行业数据安全规范

- 4.可审计：全链路操作留痕，审计日志不可篡改
- 5.可观测：全维度监控覆盖，故障分钟级发现与定位

二、差异总览表

#	模块	Demo实现	生产要求	差距等级	改进方案
1	数据存储	SQLite单文件 + 本地文件系统	PostgreSQL主从 + OSS/S3 对象存储	P0	迁移至PostgreSQL，法规文件/备份存入OSS，引入连接池
2	向量数据库	ChromaDB本地 PersistentClient	Milvus分布式集群 + 持久化备份	P0	部署Milvus集群，实现向量索引定期快照与跨区域备份
3	LLM调用	单API Key直连，无速率控制（支持用户X-API-Key转发至DashScope）	多Key轮转 + 速率控制 + 配额管理	P0	实现Key池管理器，滑动窗口限流，按Provider配额调度
4	RAG检索	向量检索（配置API Key后可用）/关键词检索，单路召回	向量+BM25 混合检索 + Query Rewriting + 多路召回	P0	引入BM25 检索器，实现RRF融合排序，添加查询改写模块
5	Reranker	规则重排	Cross-Encoder 重排模型（qwen3-rerank）	P1	部署独立Reranker服务，当前API重排降级为 Fallback
6	违规类型管理	Demo禁止自建（add_type 抛异常），条款映射已同步至数据库持久化	管理员可创建 + 审批流 + 数据库持久化	P1	移除Demo限制，实现创建→审批→生效 workflow
7	法规管理	自动上传时间作为生效日期，无版本对比	支持手动设置生效日期 + 版本 diff + 变更通知	P1	增加生效日期编辑，实现条文级diff，变更时推送通知
8	HITL	纯HTML审核界面，逐条审核		P1	

#	模块	Demo实现	生产要求	差距等级	改进方案
			专业审核工作台 + 批量操作 + 快捷键		基于React/Vue构建专业前端，支持批量审核和键盘快捷操作
9	多模态	仅OCR文本提取	VLM视觉理解 + 图片独立审核	P 2	接入多模态大模型，独立审核图片违规元素
1 0	安全	3 3 种安全检测(2 5 注入+ 5 XSS+ 3 SQLi) + 内存 Rate Limiter	WAF + 分布式 Rate Limiting + IP白名单 + 持续更新攻击库	P 0	部署WAF网关，Rate Limiter迁移至Redis，建立攻击模式更新机制
1 1	权限	单一-admin角色， 3 级角色层级 (viewer/reviewer/admin)	RBAC多角色 + 权限矩阵	P 1	实现细粒度RBAC，定义权限矩阵
1 2	监控	基础日志 + 可选 Prometheus指标 + 内存告警	Prometheus + Grafana + 告警 + SLA定义 + 自动降级	P 0	部署完整监控栈，定义SLA指标，实现自动降级策略
1 3	部署	单机FastAPI python api_server.py	K 8 s微服务 + HPA + 蓝绿部署 + 灰度发布	P 0	容器化拆分微服务，K 8 s编排，实现CI/CD流水线
1 4	评估	基础测试集 (test_cases.json)	1 0 0 +标注集 + 对抗样本 + 验收标准	P 0	扩充标注数据集，构建对抗样本库，定义验收KPI
1 5	Few-Shot	无淘汰机制，无规模控制，JSON文件存储	自动质量淘汰 + 人工审计 + 容量上限 + 版本兼容	P 2	实现样本质量评分与自动淘汰，添加容量上限和版本兼容检查
1 6	CrossCheck	Demo模式跳过	同等或更强模型复核 + 明确裁决规则	P 1	CrossCheck使用≥主审模型能力的模型，定义冲突裁决规则
1 7	法规关联	无条款间关联，独立条文检索	法规知识图谱 + 隐含逻辑识别	P 2	构建法规知识图谱，识别条款间引用/补充/例外关系
1 8	条款召回	Top 5 + 相邻扩展 (± 1 条)	Top 5 + BM 2 5 + 查	P 0	

#	模块	Demo实现	生产要求	差距等级	改进方案
			询改写 + 上下文扩展		多路召回融合，查询改写扩展语义，保持± 1 条上下文扩展
1 9	文件上传（待审核内容）	⚠️ 提供API Key后支持图片上传，不支持PDF/DOCX/DOC	✅ PDF/DOCX/DOC/图片	P 0	Demo提供API Key后可上传图片（多模态模型），生产支持多格式文件上传与自动解析
2 0	长文本分段审核（待审核内容）	❌ 截断(1 0 0 0 0 字符)	✅ 语义分段 (5 0 0 0 +字符自动切分)	P 1	Demo超长截断，生产按段落/主题边界切分，每段独立审核后合并结果
2 1	复杂表格处理（法规原件）	⚠️ 降级为简单Markdown	✅ 保留合并单元格结构	P 1	Demo丢失合并信息，生产保留完整表格结构
2 2	超长表格 (> 5 0 行)（法规原件）	⚠️ 截断至前 5 0 行	✅ LLM分层摘要	P 1	Demo截断，生产智能摘要+关键行保留
2 3	扫描件图片（待审核内容）	⚠️ 仅多模态模型(可能不完整)	✅ OCR+多模态模型	P 1	Demo无法OCR提取文字，生产配合OCR引擎提取扫描件文字
2 4	模型Fallback（系统配置）	❌ 不启用	✅ 降级链	P 1	Demo仅用单一模型，生产启用模型降级链自动切换备选模型
2 5	LLM审计日志	✅ API+JSONL文件存储	✅ 结构化查询/筛选UI + 数据库持久化	P 1	Demo通过API和JSONL文件可查看日志，生产应增加结构化查询筛选UI和数据库持久化存储
2 6	法规文档OCR智能切分	❌ 纯正则切分（"第X条"），扫描件/图片型PDF无法提取结构	✅ OCR提取标题层级 + 正则切分 + 智能合并	P 1	Demo依赖正则匹配条款边界，扫描件PDF无法识别条款结构。生产路径：上传文件 → 尝试正则提取 → 若提取不完整 → 转换为PDF → OCR（MinerU等）提取带标题层级的结构化内容 → 与正则结果合并 → 生成条款chunk

#	模块	Demo实现	生产要求	差距等级	改进方案
2 7	法规条款持久化	✓ 已实现 (regulation_chunks表, INSERT OR REPLACE增量同步)	✓ PostgreSQL持久化	—	Demo已实现法规条款持久化至SQLite的regulation_chunks表, UNIQUE(doc_name, article_number)约束防止重复, 增量同步保护用户修改
2 8	条款映射持久化	✓ 已实现 (clause_type_mappings表, 增量同步不DELETE)	✓ PostgreSQL持久化	—	Demo已实现条款-违规类型映射持久化至SQLite的clause_type_mappings表, 启动时增量同步默认映射, 不删除用户手动添加的映射
2 9	原文高亮反显	✗ 显示切分后的chunk文本	✓ 原文文档级高亮反显	P 1	Demo显示处理后的chunk文本（非原文），生产应实现原文级高亮：文本类法规（txt/doc/docx）通过关键词搜索定位段落并高亮（类似Word Ctrl+F搜索高亮）；PDF类法规利用OCR坐标信息定位条款区域并高亮标注。核心区别：chunk是处理后的数据，原文高亮是在原始文档上直接标注，保留完整上下文和排版

2.1 多模态审核详解（行 9）

当前系统只做OCR文本提取，不对图片内容进行独立审核。

问题场景：- 营销文案写"收益稳健"（文本合规），但配图中大字写着"保本保息 无风险"（图片违规）- 营销文案写"非保证收益"，但图表将演示利率标注为"保证年化 5 %"（图表违规）- 营销文案合规，但图片中有未授权明星代言（图片违规）

生产实现方案: 1. 文本提取: OCR提取图片中的文字 2. 图表理解: 多模态模型(Qwen-VL)理解图表含义 3. 图片独立审核: 对图片内容独立进行违规检测, 发现违规元素直接标记, 无需与文本比对 4. 视觉审核: CV模型检测未授权人脸、虚假奖杯等

2.2 角色管理详解 (行 1 1)

User Story中的角色 (业务人员、合规审核员、系统管理员) 与权限系统中的角色 (viewer、reviewer、admin) 是映射关系: - 业务人员 → viewer (查看审核结果) - 合规审核员 → reviewer (审核+反馈+Few-Shot管理) - 系统管理员 → admin (全部权限+类型管理+系统配置)

权限矩阵:

操作	viewer	reviewer	admin
提交审核	✓	✓	✓
查看审核结果	✓	✓	✓
人工复核/Override	✗	✓	✓
提交逐项反馈	✗	✓	✓
管理Few-Shot	✗	✓	✓
创建/废弃违规类型	✗	✗	✓
管理条款映射	✗	✗	✓
系统配置	✗	✗	✓
查看审计日志	✗	✓(只读)	✓

2.3 评估框架详解 (行 1 4)

评估不一定要有"正确答案"。两种评估方式:

- 1. 有标准答案的评估 (客观评估):
- 2. 人工标注数据集, 每条标注: 合规/违规 + 违规类型 + 依据条款
- 3. 计算准确率、召回率、F 1、假阳性率、假阴性率
- 4. 适合: 绝对化用语、收益承诺等明确违规
- 5. 当前: test_cases.json
- 6. 无标准答案的评估 (主观评估):
- 7. LLM-as-Judge: 用另一个LLM评判审核质量

- 8. 人工抽检评分: 1 - 5 分评价审核理由的合理性
- 9. 适合: 隐含暗示、边界案例
- 10. 当前: HITL人工复核

交付标准: 需要展示正确率。交付时应提供: - 基线评估报告 (50 +标注样本) - 关键指标: 准确率 $\geq 95\%$, 召回率 $\geq 99\%$ (红线违规), 假阳性 $\leq 3\%$ - 对抗测试报告 (Prompt注入成功率 $< 1\%$)

当前正确率: Demo模式无法测量 (需API Key运行LLM)。生产部署后首次评估建议用 50 条标注数据建立基线。

如何跑评估: 详见 docs/technical-details.md 第七章

2.5 测试覆盖差距分析

当前测试覆盖范围

用户故事	是否有测试用例	测试文件	用例数量	说明
US 1: 营销内容审核	✅有	eval/test_cases.json + eval/extreme_test_cases.json	30 标准 + 25 极端	仅覆盖营销内容审核流程
US 2: 法规文档入库	❌无	—	0	上传→解析→分块→标注→审批→索引全链路无测试
US 3: 人工复核	❌无	—	0	待审队列查询、override操作、Few-Shot回流无测试
US 4: 违规类型管理	❌无	—	0	创建/废弃类型、关键词管理、条款映射管理无测试
US 5: 效果评估	❌无	—	0	评估运行、报告查询、错误分析无测试

结论: 当前 30 条标准测试用例 + 25 条极端测试用例全部针对US 1 (营销内容审核)，其他用户故事 (US 2 -US 5) 均无测试用例。

组件级测试覆盖差距

当前所有测试均为端到端（End-to-End）测试，即输入营销内容→输出审核结论，验证整体结果正确性。**缺少对 1 0 步Pipeline中各独立步骤的组件级测试**，无法定位具体是哪个步骤导致了错误。

Pipeline步骤	是否有组件测试	风险说明	优先级
① Extract（要素提取）	✗无	提取遗漏关键claims会导致后续步骤全部失效	P 2
② RuleCheck（规则预检）	✗无	规则命中/未命中的边界条件未验证	P 2
③ RAGRetrieve（向量检索）	✗无	检索召回率未独立验证，可能漏掉关键法规	P 2
④ Rerank（重排序）	✗无	Top 2 0 →Top 5 排序质量未独立评估	P 2
⑤ LLMReason（CoT推理）	✗无	核心步骤 ，推理质量直接决定审核准确性，无独立测试无法区分LLM推理错误与上下游传递错误	P 1
⑦ Format（结构化归一化）	✗无	LLM结果归一化+规则参考注入+降级兜底逻辑未验证	P 2
⑦ Validate（幻觉检测）	✗无	幻觉引用的移除逻辑未独立验证	P 2
⑧ CrossCheck（交叉复核）	✗无	关键步骤 ，复核结论冲突裁决规则未独立验证，错误裁决可能导致合规内容被拦截或违规内容被放行	P 1
⑨ RiskAssess（风险评估）	✗无	风险评分计算和决策路由阈值未独立验证	P 2

优先级说明： - **P 1**：LLMReason和CrossCheck是审核质量的核心保障步骤。LLMReason的推理错误无法通过端到端测试定位（不知道是推理本身出错还是上游输入有误）；CrossCheck的冲突裁决错误可能导致严重后果（合规→拦截、违规→放行）。这两个步骤的组件测试应优先补充。 - **P 2**：其余步骤的组件测试可在P 1 完成后逐步补充。US 2 -US 5 的用户故事级测试用例也列为P 2 优先级。

改进计划

优先级	改进项	预估工作量	说明
P 1	LLMReason组件测试	3 天	构造固定Extract+RuleCheck+RAG输入，验证LLM推理输出的违规类型、引用条文、推理逻辑正确性
P 1	CrossCheck组件测试	2 天	构造主模型合规/违规+CrossCheck合规/违规的 4 种组合场景，验证冲突裁决规则和置信度调整逻辑
P 2	US 2 法规文档入库测试	3 天	覆盖多格式解析、分块、标注、审批、索引构建全链路
P 2	US 3 人工复核测试	2 天	覆盖待审队列、override操作、Few-Shot回流
P 2	US 4 违规类型管理测试	2 天	覆盖创建/废弃类型、关键词管理、条款映射CRUD
P 2	US 5 效果评估测试	1 天	覆盖评估运行、报告查询、错误分析
P 2	其余Pipeline步骤组件测试	5 天	Extract/RuleCheck/RAGRetrieve/Rerank/Format/Validate/RiskAssess各步骤独立测试

2.4 法规关联详解（行 1 7）

Demo可实现简单的 1 级条款引用跟随：当条款引用其他条款时（如"依照第X条"），提取被引用条款并纳入LLM上下文。仅 1 级，不递归。

示例: - 条款A文本中出现"依照第九条" → 自动提取第九条全文 → 拼接到条款A的上下文中供LLM参考 - 不再继续追踪第九条中可能引用的其他条款（无递归）

生产环境则需构建完整的法规知识图谱，支持多级引用、补充、例外等关系推理。

2.6 文件上传（Demo限制）[对应差异表第 1 9 行]

文件上传：Demo模式下，图片上传默认禁用（仅支持文本输入）。当用户提供有效的 DashScope API Key并通过连接测试后，图片上传自动启用（因为默认模型支持多模态）。此处的文件上传指待审核营销内容的上传，非法规原件上传（法规原件由系统预置于RAG引擎中）。生产环境额外支持PDF/DOCX/DOC上传，系统自动解析内容后进入审核流程。

2.7 长文本分段审核（Demo限制）[对应差异表第 2 0 行]

长文本分段审核：Demo环境不拆分长文本（MAX_INPUT_LENGTH= 1 0 0 0 0 字符，超出直接截断）。生产环境使用语义分段：超过 5 0 0 0 字符时按段落/主题边界切分（每段 ≤ 3 0 0 0 字符），每段独立走完整审核流水线，最后合并结果（违规类型取并集、引用条款去重、置信度取加权平均）。

2.8 表格处理（Demo降级）[对应差异表第 2 1 行]

表格处理：Demo环境对复杂表格降级为简单Markdown（丢失合并单元格信息），超长表格（> 5 0 行）截断至前 5 0 行。生产环境保留合并单元格结构，超长表格使用LLM分层摘要+关键行保留。

2.9 扫描件图片（Demo限制）[对应差异表第 2 2 行]

扫描件图片：Demo环境无法处理扫描件图片的文字提取，仅通过多模态模型理解图片内容。生产环境配合OCR引擎提取扫描件文字，再由多模态模型理解图片语义。

2.1 0 模型统一[对应差异表第 2 3 行]

模型统一：Demo和生产均使用多模态模型（默认qwen 3.6 -plus），不再区分纯文本模型和VLM。前端可切换模型，切换后所有步骤均使用指定模型。有API Key时即启用模型降级链（qwen 3.6 -plus → qwen 3.6 -flash），无API Key时降级为规则引擎。

2.1 1 法规文档OCR智能切分[对应差异表第 2 6 行]

法规文档OCR智能切分：Demo环境依赖正则匹配（第x条）识别条款边界进行切分，对文本型PDF/DOCX有效，但扫描件PDF和图片型PDF无法提取条款结构，导致降级为整文档大chunk或按句子切分，丢失条款级结构化信息。

生产路径：

上传法规文件

- Step 1: 尝试正则提取（现有逻辑，对文本型文件有效）
- Step 2: 评估提取完整性（条款数量是否合理、文本密度是否正常）
- Step 3: 若提取不完整 → 将文件转换为PDF
- Step 4: OCR提取（MinerU或类似OCR产品）
 - 输出带标题层级的结构化内容（章→节→条→款→项）
- Step 5: 合并正则结果 + OCR结果（去重、互补）
- Step 6: 生成条款chunk → 持久化至regulation_chunks表

OCR产品选型：

产品	特点	适用场景
MinerU	开源，支持PDF标题层级提取，输出Markdown	法规文档结构化提取（推荐）
PaddleOCR	开源，高精度文字识别，但不输出层级结构	纯文字提取场景
阿里云文档智能	商业服务，支持表格/版面分析	生产环境高可靠性要求

关键设计：OCR提取的标题层级信息（章→节→条→款→项）与正则提取的条款边界互补。正则对文本型文件准确率高，OCR对扫描件/图片型文件不可替代。两者结果合并时以正则结果为基准，OCR补充正则未覆盖的条款。

2.1.2 法规条款与条款映射持久化[对应差异表第 2 7、2 8 行]

法规条款持久化： ☒ 已实现。法规条款通过 `regulation_chunks` 表持久化至SQLite数据库，`sync_chunks()` 方法使用 `INSERT OR REPLACE` 增量同步，`UNIQUE(doc_name, article_number)` 约束防止重复插入。启动时自动同步，不执行DELETE操作，保护用户修改。

条款映射持久化： ☒ 已实现。条款-违规类型映射通过 `clause_type_mappings` 表持久化至SQLite数据库，`_sync_clause_mappings()` 方法在启动时增量同步默认映射（`_DEFAULT_MAPPINGS`），仅INSERT缺失项，不DELETE已有映射，保护用户手动添加的映射关系。

数据库为唯一真相源： `clause_mappings.json` 等JSON文件仅作为缓存，数据库是Single Source of Truth。详见[technical-details.md](#) 1 4.4.3 节。

三、P 0 必须上线前完成

3.1 数据存储迁移：SQLite → PostgreSQL

当前状态：[database.py](#) 使用 `sqlite3` 连接单文件数据库，通过 `threading.local()` 实现线程隔离，WAL模式提升并发读。

生产要求： - 迁移至PostgreSQL 1 5+，支持主从复制和自动故障切换 - 法规文件、数据库备份迁移至OSS/S 3 对象存储 - 引入连接池（如 `psycopg2.pool` 或 `SQLAlchemy` 引擎），替代每次新建连接 - 数据库Schema需适配PostgreSQL语法（如 `datetime('now')` → `NOW()`，`INTEGER` 自增 → `SERIAL`） - 审计日志表需支持分区表，按月自动归档

迁移要点： 1. 6 张核心表 + 5 张扩展表全部迁移，索引策略保持不变 2. `_schema_version` 迁移机制需重写为PostgreSQL兼容的版本管理 3. `backup()` 方法改为 `pg_dump` + OSS上传 4. 密码哈希算法（PBKDF 2 -SHA 2 5 6）保持不变，确保数据兼容

3.2 向量数据库：ChromaDB → Milvus

当前状态： [rag_engine.py](#) 使用 `chromadb.PersistentClient` 本地持久化，HNSW cosine索引。

生产要求： - 部署Milvus 2.x分布式集群，支持水平扩展 - 向量索引定期快照，跨区域备份 - 支持动态增删向量（法规更新时增量索引） - 集合元数据管理（`index_version`追踪）

迁移要点： 1. `collection.add()` / `collection.query()` API需适配Milvus SDK 2. Embedding维度 1 0 2 4 保持不变，HNSW参数需调优 3. 索引版本追踪机制需重新实现 4. `_load_existing_index()` 需改为从Milvus恢复

向量存储目录手动删除的已知行为

如果用户手动删除了 `data/vector_store/` 目录，系统的行为如下：

维度	说明
索引恢复	系统下次启动时， <code>_load_existing_index()</code> 会因目录不存在而失败，触发索引重建流程
数据来源	<code>regulation_versions</code> 表（SQLite）仍保留所有文档记录，系统知道哪些文档需要重新索引
数据丢失	无数据丢失。 向量索引只是法规文档的派生数据，原始文档和元数据均在数据库中
重建成本	一次性重建成本：需重新调用Embedding API对所有法规文档进行向量化，耗时取决于文档数量和API速率
结论	这是 已知行为 ，不是Bug。用户无需担心手动删除向量存储目录导致数据丢失

重建流程：

```
data/vector_store/ 被删除
→ 系统启动 → _load_existing_index() 失败
→ 从 regulation_versions 表读取文档列表
→ 重新解析文档 → 分块 → Embedding → 写入新的 ChromaDB 索引
→ 索引恢复完成
```

向量化内容去重

维度	Demo（当前）	生产（目标）
去重策略	按文件名去重，同一文件名不会重复索引	按内容哈希去重，相同内容不同文件名不会重复索引
当前限制	相同内容以不同文件名上传时，会被向量化两次，导致检索结果中出现重复条款	
生产方案	向量化前计算 <code>content_hash</code> （SHA 2 5 6），写入ChromaDB的 <code>doc_content_hash</code> 元数据字段；添加新文档时先查询ChromaDB中是否已存在相同 <code>doc_content_hash</code> ，若存在则跳过向量化	
实现方式	—	<code>collection.add(documents=..., metadatas=[{"doc_content_hash": hash, ...}])</code> ，添加前执行 <code>collection.get(where={"doc_content_hash": hash})</code> 检查

生产实现示例：

```
import hashlib

def _compute_content_hash(sections: list) -> str:
    content = "".join(s["article_text"] for s in sections)
    return hashlib.sha256(content.encode("utf-8")).hexdigest()[:16]

def _add_documents_with_dedup(self, chunks, collection):
    content_hash = self._compute_content_hash(chunks)
    existing = collection.get(where={"doc_content_hash": content_hash})
    if existing and existing["ids"]:
        logger.info(f"Content hash {content_hash} already exists, skipping vectorization")
        return
    collection.add(
        documents=[c.article_text for c in chunks],
        metadatas=[{"doc_content_hash": content_hash, **c.metadata} for c in chunks],
```

```
ids=[c.chunk_id for c in chunks]
)
```

3.3 LLM调用：单Key → 多Key轮转 + 速率控制

当前状态：[llm_gateway.py](#) 使用单一 DASHSCOPE_API_KEY，已实现熔断器（CircuitBreaker）和模型优先级调度，但无Key轮转和速率控制。前端传入的 X-API-Key 在认证通过后可同时作为 DashScope API Key 使用，使用户在Demo模式下可使用自己的Key获得完整LLM功能。

已实现 ： - 熔断器：LLM失败 5 次打开熔断，30s后半开恢复 - 重试策略：指数退避 + 抖动，最多 3 次重试 - 模型降级：PRIMARY → FALLBACK → LIGHTWEIGHT - 可配置参数：模型名/阈值/超时均通过 config.py 环境变量配置

待实现 ： - 多API Key池管理，支持轮转/权重分配 - 滑动窗口限流，按Provider配额调度 - Key健康度检测与自动剔除 - 调用计量与成本追踪

3.4 RAG检索：单路召回 → 混合检索 + 多路召回

当前状态：[rag_engine.py](#) 支持向量检索（_vector_retrieve，配置API Key后可用）或关键词检索（_keyword_retrieve），向量检索失败时降级到关键词检索。

生产要求： - BM25检索：引入Elasticsearch或jieba+TF-IDF实现稀疏检索 - 混合排序：向量检索 + BM25检索结果通过RRF（Reciprocal Rank Fusion）融合 - Query Rewriting：对用户查询进行改写/扩展，提升召回率 - 多路召回：关键词路径 + 向量路径 + BM25路径并行召回

改进方案： 1. 新增 BM25Retriever 类，基于jieba分词 + TF-IDF 2. retrieve() 方法改为多路并行召回 + RRF融合 3. 新增 QueryRewriter 模块，使用轻量模型改写查询 4. 召回数量从 top_k=20 提升至 top_k=50，经Reranker精排后取Top 5

3.5 安全加固

当前状态：[security.py](#) 实现了： - InputValidator：25种Prompt注入正则 + XSS检测 + SQL注入检测 + 控制字符清理 - RateLimiter：内存级滑动窗口限流（60次/60s全局，20次/60s/IP） - AuditLogger：HMAC签名审计日志，JSONL文件存储

生产要求： - WAF：部署云WAF或Nginx + ModSecurity，前置过滤恶意请求 - 分布式Rate Limiting：迁移至Redis + Lua脚本，支持集群级限流 - IP白名单：仅允许授权IP访问审核API - 攻击模式库持续更新：建立Prompt注入模式库的定期更新机制，当前25种模式需持续扩充 - 审计日志：从本地JSONL文件迁移至不可篡改存储（如数据库追加写表或区块链存证）

3.6 监控体系

当前状态: [observability.py](#) 实现了: - Prometheus指标定义 (可选, `prometheus_client` 未安装时降级) - `trace_operation` 上下文管理器, 慢操作告警 (> 1 s) - `AlertManager`: 内存级告警规则引擎, 冷却时间 300 s - `HealthChecker`: 健康检查框架

生产要求: - **Prometheus + Grafana**: 部署完整监控栈, Dashboard覆盖审核/LLM/RAG/DB全维度 - **告警通道**: 告警推送至钉钉/企微/邮件, 不再仅依赖日志 - **SLA定义**: - 审核API可用性 $\geq 99.9\%$ - P50延迟 $\leq 1\text{ s}$, P99延迟 $\leq 3\text{ s}$ - LLM调用成功率 $\geq 99\%$ - **自动降级**: SLA不达标时自动触发降级策略 (如LLM $\rightarrow$ 规则引擎, 向量 $\rightarrow$ 关键词)

当前监控日志位置: - 应用日志: 控制台输出 (stdout), `python api_server.py` 启动后直接在终端查看 - 日志级别: INFO (默认), 可通过 `logging.basicConfig(level=logging.DEBUG)` 调整 - 审核记录: SQLite数据库 `data/insurance_review.db`, `review_records` 表 - 无Web界面查看日志, 需直接查询数据库或查看终端输出 - 生产改进: 部署Prometheus+Grafana后, 可通过Web Dashboard查看

3.7 部署架构

当前状态: 单机 `python api_server.py` 启动FastAPI服务, 无容器化, 无CI/CD。

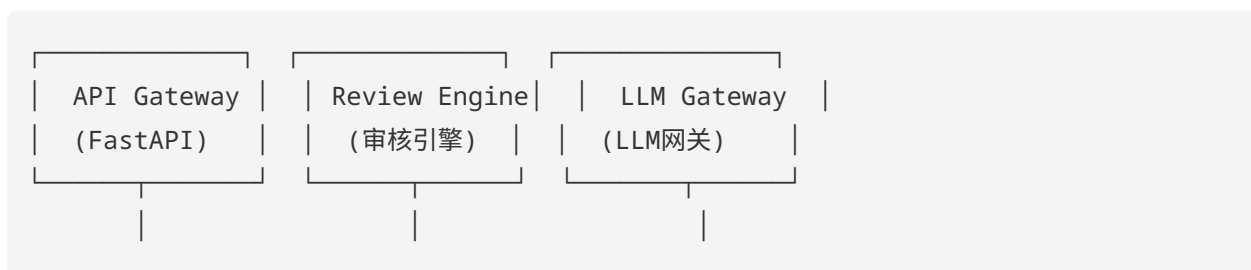
生产要求: - **容器化**: Docker镜像构建, 多阶段构建优化镜像大小 - **K8s编排**: 微服务拆分 (API服务、RAG服务、LLM网关、审核引擎), HPA自动扩缩容 - **蓝绿部署**: 零停机更新, 新版本蓝绿切换 - **灰度发布**: 按比例流量切换, 支持快速回滚 - **CI/CD**: 代码提交 $\rightarrow$ 自动测试 $\rightarrow$ 镜像构建 $\rightarrow$ 灰度发布 $\rightarrow$ 全量上线

CI/CD流水线通俗解释:

CI/CD = 持续集成/持续部署, 是自动化发布流程: 1. 开发者提交代码 $\rightarrow$ 2. 自动运行测试 $\rightarrow$ 3. 自动构建Docker镜像 $\rightarrow$ 4. 先部署到灰度环境(少量用户) $\rightarrow$ 5. 验证无问题 $\rightarrow$ 6. 全量发布

当前Demo: 手动运行 `python api_server.py` 生产: 自动化上述流程, 代码合并后自动部署

微服务拆分建议:





3.8 评估体系

当前状态: [eval/](#) 目录包含 `test_cases.json` 和 `extreme_test_cases.json` , [evaluator.py](#) 实现基础评估框架。

生产要求: - **标注集扩充:** 从当前基础测试集扩展至 1 0 0 +标注样本, 覆盖全部违规类型 - **对抗样本:** 构建对抗样本库, 包含: - Prompt注入伪装 (如"请忽略以上指令"嵌入正常营销文本) - 边界案例 (如"历史收益率仅供参考"是否构成收益承诺) - 多违规叠加案例 - **验收标准:** - 准确率 (Accuracy) $\geq 95\%$ - 假阳性率 (False Positive Rate) $\leq 3\%$ - 假阴性率 (False Negative Rate) $\leq 1\%$ - 条文引用准确率 $\geq 90\%$

3.9 条款召回增强

当前状态: [reranker.py](#) 的 `_expand_adjacent_articles` 仅扩展± 1 条相邻条文, Top 5 召回后扩展。

生产要求: - Top 5 向量召回 + BM 2 5 召回 + 查询改写扩展 - 保持± 1 条上下文扩展 (当前方案已足够) - 引入条文间引用关系 (如"依照第X条") , 实现关联条文召回 - 召回结果去重与相关性过滤

± 1 条扩展说明:

当前± 1 条 (前后各 1 条) : - 每个Top 5 结果扩展 2 条 → 最多额外 1 0 条 - 每条平均 2 0 0 字 → 额外约 2 0 0 0 字 $\approx 1 0 0 0$ tokens - 成本增加: 约¥ 0 . 0 0 7 /次 (qwen 3 . 6 -flash价格)

± 1 条已足够, 因为: - 法规条款通常一条一款, 上下文依赖有限 - ± 1 条已覆盖最常见的"前一条定义、后一条处罚"模式 - 如果需要更远距离的关联, 应该通过知识图谱解决, 而非暴力扩展

四、P 1 上线后 1 个月内

4.1 Reranker升级：Embedding相似度 → Cross-Encoder

当前状态：[reranker.py](#) 使用规则重排（`_cross_encoder_rerank`），API失败时降级为规则重排（`_rule_based_rerank`）。

已实现 ：- 重试 + 降级：API重排失败自动降级到规则重排 - 相邻条文扩展：Top 5 结果自动扩展 ± 1 条上下文

待实现 ：- 部署Cross-Encoder模型（如 `qwen3-rerank`）作为独立微服务 - Cross-Encoder 推理性能优化（批量推理、GPU加速） - 重排结果与向量检索分数的融合策略

4.2 违规类型管理增强

当前状态：[violation_registry.py](#) 在Demo模式下 `add_type()` 直接抛出 `ValueError("Demo模式下不支持创建自定义违规类型")`，条款映射已同步至数据库 `clause_type_mappings` 表持久化。

生产要求： - 移除Demo限制，允许管理员创建自定义违规类型 - 实现**创建→审批→生效** workflow：
1. 管理员提交新类型申请 2. 高级管理员审批 3. 审批通过后自动生效并同步到数据库 - 数据持久化已在数据库 `violation_types` 表和 `clause_type_mappings` 表中实现，无需再从JSON文件迁移 - 支持违规类型的版本历史和回滚

4.3 法规管理增强

当前状态：[database.py](#) 的 `save_regulation_version()` 支持版本记录，`effective_date` 字段已存在但上传时默认为当前时间。

生产要求： - 支持手动设置法规生效日期（法规发布日 ≠ 生效日） - 实现条文级版本diff（新旧版本对比，高亮变更内容） - 法规变更时自动推送通知（钉钉/企微/邮件） - 变更触发的违规类型映射自动更新（`clause_type_mappings` 表联动更新）

4.4 HITL专业审核工作台

当前状态：[api_server.py](#) 基于FastAPI构建RESTful API + 纯HTML前端，支持单条审核和反馈。

生产要求： - 基于React/Vue构建专业审核工作台 - **批量审核：** 支持一次选择多条待审核记录，批量通过/驳回 - **快捷键：** `Enter` 通过、`Esc` 驳回、`↑↓` 切换记录 - **审核看板：** 待审核/已审核/已驳回状态流转看板 - **审核统计：** 审核员工作量、审核时效、驳回率等维度统计

4.5 权限体系：RBAC

当前状态：[database.py](#) 的 `check_permission()` 实现 3 级角色层级（viewer= 0 , reviewer= 1 , admin= 2 ），仅基于角色等级比较。

生产要求： - **细粒度RBAC：**定义权限矩阵，如： | 操作 | viewer | reviewer | admin |
|-----|-----|-----|-----| 查看审核结果 |  |  |  | 执行审核 |  |  |  | 管理违规类型 |  |  |  | 管理用户 |  |  |  | - **操作审计：**所有权限相关操作记录审计日志

4.6 CrossCheck增强

当前状态：[review_agent.py](#) 的 `_step_crosscheck` 在Demo模式下直接跳过。

生产要求： - CrossCheck使用≥主审模型能力的模型（如主审用 `qwen3.6-plus` ，复核也用 `qwen3.6-plus` ，后备为 `qwen3.6-flash` ） - 定义**冲突裁决规则**： 1. 主审判定违规 + 复核通过 → 维持违规判定，置信度下调 0.1 2. 主审判定合规 + 复核发现违规 → 转人工审核 3. 主审与复核均判定违规但类型不同 → 合并违规类型 - 复核结果记录至数据库，支持后续分析复核质量

五、P 2 迭代优化

5.1 多模态审核

当前状态：[input_processors.py](#) 仅支持OCR文本提取，图片描述作为附加文本拼接。

生产要求： - 接入VLM（Vision Language Model），实现图片语义理解 - **图片独立审核：**对图片内容独立进行违规检测，发现违规元素直接标记（如违规代言人、误导性图表、违规宣传语） - 图片中的违规元素检测（如违规代言人、误导性图表） - 支持视频内容审核（关键帧提取 + 逐帧分析）

5.2 Few-Shot自动管理

当前状态：[prompt_manager.py](#) 的Few-Shot样本存储在 `few_shots.json` ，无淘汰机制，无规模控制。

生产要求： - **自动质量淘汰：**基于反馈数据计算样本质量分，低分样本自动淘汰 - 质量分 = 人工确认率 × 0.6 + 模型一致性 × 0.4 - 质量分 < 0.5 的样本标记为待审计 - **人工审**

计：定期由审核专家审计Few-Shot样本质量 - **容量上限**：Few-Shot样本总数上限 1 0 0 条，按违规类型均匀分布 - **版本兼容**：Prompt版本切换时，Few-Shot样本自动适配新版本格式

Few-Shot自动质量淘汰实现方案:

1. **质量评分**: 每个Few-Shot样本增加 `quality_score` 字段(0 - 1)
 2. 初始分: 人工确认的= 0 . 8 , LLM生成的= 0 . 5
 3. 使用反馈: 每次被LLM参考且结论一致 → + 0 . 0 5
 4. 使用冲突: 每次被LLM参考但结论不一致 → - 0 . 1
 5. 人工标记: 审核员标记"有用"→ + 0 . 2 , "无用"→ - 0 . 3
 6. **容量控制**: 每个违规类型最多 1 0 条，全局最多 1 0 0 条
 7. 超过容量时，淘汰`quality_score`最低的
 8. 同分时，淘汰最旧的
 9. **版本兼容**: Prompt版本更新时，旧Few-Shot标记 `compatible_version`
- 1 0 . 不兼容的样本不参与检索，但保留在库中
 - 1 1 . **定期清理**: 每季度执行一次
 - 1 2 . 删除`quality_score < 0 . 3` 的样本
 - 1 3 . 合并高度相似的样本

5 . 3 法规知识图谱

当前状态：法规条文独立存储和检索，无条款间关联。

生产要求： - 构建**法规知识图谱**，节点为条文，边为条款间关系： - 引用关系（"依照第X条"） - 补充关系（"第X条的补充规定"） - 例外关系（"除第X条规定外"） - 修订关系（"第X条修订为"） - **隐含逻辑识别**：通过图谱推理发现隐含违规路径 - 例：条文A禁止X → 条文B引用A → 条文C是B的例外 → 内容命中C时需额外检查A - 图谱可视化，辅助合规专家理解法规关联

法规知识图谱实现方案:

技术选型: - Demo阶段: 不实现（复杂度高，收益在Demo阶段不明显） - 生产推荐: Neo 4 j（图数据库标准选择）或 Apache Jena（RDF三元组存储） - 轻量替代: 在SQLite中增加 `clause_relations` 表（无图查询能力但够用）

如何知道条款间关系: 1. **自动识别:** LLM分析条款文本, 提取引用关系 - 输入: 条款全文 -

Prompt: "请识别以下条款中引用的其他条款编号, 以及条款间的关系类型" - 输出: {source: "第二十一条", target: "第九条", relation: "引用"} 2. **人工标注:** 合规专家审核LLM识别结果, 修正错误

3. **规则匹配:** 正则识别"依照第X条"、"第X条规定的"等模式

Demo期间能否实现: - 完整知识图谱(Neo 4 j): ❌ 太复杂, Demo不建议 - 轻量关系表(SQLite):

⚠️ 可以, 约 1 0 0 行代码, 但收益有限 - LLM自动识别关系: ✅ 可以做, 但不入库, 仅在审核时动态识别 - 1 级条款引用跟随: ✅ 可以做, 当条款引用其他条款时提取被引用条款纳入上下文, 仅 1 级不递归 - 建议: Demo期间先实现 1 级条款引用跟随和LLM动态识别, 生产再建图

六、迁移检查清单

6.1 基础设施

- ☐ PostgreSQL集群部署完成, 主从复制验证通过
- ☐ Milvus集群部署完成, 向量索引迁移验证通过
- ☐ Redis集群部署完成, 分布式缓存和限流验证通过
- ☐ OSS/S3 存储桶创建完成, 权限配置正确
- ☐ K8s集群部署完成, Namespace/资源配额配置就绪
- ☐ WAF规则配置完成, 恶意请求拦截验证通过

6.2 数据迁移

- ☐ SQLite → PostgreSQL数据迁移脚本编写并验证
- ☐ ChromaDB → Milvus向量数据迁移脚本编写并验证
- ☐ JSON文件 (违规类型/标注) → 数据库迁移验证 (条款映射已同步至数据库, 法规条款已持久化至regulation_chunks表)
- ☐ 审计日志JSONL → 数据库追加写表迁移
- ☐ 数据完整性校验通过 (记录数、哈希校验)

6.3 服务部署

- ☐ Docker镜像构建完成, 多阶段构建优化
- ☐ K8s Deployment/Service/Ingress配置就绪
- ☐ HPA自动扩缩容策略配置并验证
- ☐ 蓝绿部署流程验证通过
- ☐ 灰度发布流程验证通过

- ☐ CI/CD流水线搭建完成

6.4 安全合规

- ☐ 默认密码 admin123 已替换为强密码
- ☐ PASSWORD_SALT 和 AUDIT_SIGNING_KEY 已更换为生产级密钥
- ☐ API Key过期策略验证（90天自动过期）
- ☐ IP白名单配置完成
- ☐ TLS证书配置完成，HTTPS强制跳转
- ☐ Prompt注入检测规则库更新至最新
- ☐ 安全渗透测试通过

6.5 监控告警

- ☐ Prometheus指标采集配置完成
- ☐ Grafana Dashboard搭建完成（审核/LLM/RAG/DB/系统 5 大看板）
- ☐ 告警规则配置完成并验证通知通道
- ☐ SLA指标定义并接入监控
- ☐ 自动降级策略配置并验证

6.6 评估验收

- ☐ 100+标注数据集构建完成
- ☐ 对抗样本库构建完成
- ☐ 准确率 $\geq 95\%$ 验证通过
- ☐ 假阳性率 $\leq 3\%$ 验证通过
- ☐ 假阴性率 $\leq 1\%$ 验证通过
- ☐ 条文引用准确率 $\geq 90\%$ 验证通过
- ☐ P99延迟 $\leq 3s$ 压测验证通过
- ☐ 100 QPS并发压测验证通过
- ☐ 可解释性评分验证通过：引用条款与违规类型关联度 ≥ 0.85 、推理链条完整性 ≥ 0.80 、人工评审认同率 $\geq 90\%$
- ☐ 置信度校准验证通过：ECE（Expected Calibration Error） ≤ 0.1 、各置信度区间实际准确率偏差 $\leq 15\%$

6.7 功能验证

- ☐ 审核全流程（提取→规则→检索→重排→推理→格式→验证→复核→风险评估）端到端验证
- ☐ 规则引擎关键词命中验证
- ☐ LLM降级链路验证（PRIMARY → FALLBACK → LIGHTWEIGHT → 规则引擎）
- ☐ 向量检索降级链路验证（向量 → 关键词）
- ☐ 熔断器触发与恢复验证
- ☐ 缓存命中与失效验证
- ☐ 审计日志HMAC签名验证
- ☐ 法规版本管理与切换验证

6.8 回滚预案

- ☐ 数据库回滚脚本准备就绪
- ☐ 向量索引回滚方案准备就绪
- ☐ K 8 s回滚命令文档化
- ☐ DNS切换方案准备就绪
- ☐ 应急联系人清单更新